

Code Explanation

Batch Processing Pipeline Overview

The provided code is a batch processing pipeline implemented using Apache Spark. It is designed to process customer and order data from source CSV files, clean and transform the data, and then write it to Delta tables. The pipeline also creates an SCD Type 2 order summary table and a customer aggregate spend table.

The pipeline is divided into several functions, each responsible for a specific task. The main function, `run_pipeline``, orchestrates the entire process.

Step 1: Create a Spark Session

This step creates a Spark session, which is the entry point to any Spark functionality. The `create_spark_session`` function is used to create a Spark session with Hive support enabled.

```
def create_spark_session():  
    """Create a Spark session for batch processing."""  
    return SparkSession.builder  
        .appName("Customer Order Batch Processing")  
        .enableHiveSupport()  
        .getOrCreate()
```

Step 2: Process Customer Data

This step processes the customer data from the source CSV file. It defines a schema for the customer data, reads the data into a DataFrame, cleans the data by removing null values and duplicates, and then writes the cleaned data to a Delta table.

```
def process_customer_data(spark):  
    """Process customer data from source to silver layer."""  
    # Define schema  
    customer_schema = StructType([  
        StructField("CustId", StringType(), True),  
        StructField("Name", StringType(), True),  
        StructField("EmailId", StringType(), True),  
        StructField("Region", StringType(), True)  
    ])  
  
    # Read source data  
    customer_df = (  
        spark.read.format("csv")  
        .option("header", "true")  
        .schema(customer_schema)  
        .load(CUSTOMER_DATA_PATH)  
    )  
  
    # Clean data  
    customer_df_clean = (  
        customer_df  
        .filter(  
            (F.col("CustId").isNotNull()) &
```

```

        (F.col("Name").isNotNull()) &
        (F.col("EmailId").isNotNull()) &
        (F.col("Region").isNotNull())
    )
    .dropDuplicates(["CustId"])
)

# Write to Delta table
customer_df_clean.write.format("delta")
    .mode("overwrite")
    .saveAsTable(f"{TARGET_CATALOG}.{TARGET_SCHEMA}.{CUSTOMER_TABLE}")

return customer_df_clean

```

Step 3: Process Order Data

This step processes the order data from the source CSV file. It defines a schema for the order data, reads the data into a DataFrame, cleans the data by removing null values and duplicates, adds a new column for the total amount, and then writes the cleaned data to a Delta table.

```

def process_order_data(spark):
    """Process order data from source to silver layer."""
    # Define schema
    order_schema = StructType([
        StructField("OrderId", StringType(), True),
        StructField("ItemName", StringType(), True),
        StructField("PricePerUnit", DoubleType(), True),
        StructField("Qty", IntegerType(), True),
        StructField("Date", DateType(), True),
        StructField("CustId", StringType(), True)
    ])

    # Read source data
    order_df = (
        spark.read.format("csv")
        .option("header", "true")
        .schema(order_schema)
        .load(ORDER_DATA_PATH)
    )

    # Clean data and add TotalAmount
    order_df_clean = (
        order_df
        .filter(
            (F.col("OrderId").isNotNull()) &
            (F.col("ItemName").isNotNull()) &
            (F.col("PricePerUnit").isNotNull()) &
            (F.col("Qty").isNotNull()) &
            (F.col("Date").isNotNull()) &
            (F.col("CustId").isNotNull())
        )
        .withColumn("TotalAmount", F.col("PricePerUnit") * F.col("Qty"))
        .dropDuplicates(["OrderId"])
    )

```

```

# Write to Delta table
order_df_clean.write.format("delta")
    .mode("overwrite")
    .saveAsTable(f"{TARGET_CATALOG}.{TARGET_SCHEMA}.{ORDER_TABLE
}")

return order_df_clean

```

Step 4: Create or Update SCD Type 2 Order Summary

This step creates or updates the SCD Type 2 order summary table. It joins the customer and order data, checks if the table exists, and then either initializes the table or implements SCD Type 2 logic for updates.

```

def create_order_summary_scd2(spark, customer_df, order_df):
    """Create or update the SCD Type 2 order summary table."""
    # Check if the table exists
    table_exists = spark._jsparkSession.catalog().tableExists(TARGET_CATALOG, TARGET_SCHEMA, ORDER_SUMMARY_TABLE)

    # Join customer and order data
    joined_df = (
        order_df
        .join(customer_df, "CustId", "inner")
        .select(
            customer_df["CustId"],
            customer_df["Name"],
            customer_df["EmailId"],
            customer_df["Region"],
            order_df["OrderId"],
            order_df["ItemName"],
            order_df["PricePerUnit"],
            order_df["Qty"],
            order_df["Date"],
            order_df["TotalAmount"]
        )
    )

    # ... (rest of the function remains the same)

```

Step 5: Create Customer Aggregate Spend

This step creates the customer aggregate spend table. It reads from the order summary table, aggregates the total amount by customer name and date, and then writes the aggregated data to a Delta table.

```

def create_customer_aggregate_spend(spark):
    """Create the customer aggregate spend table."""
    # Read from the order summary table, only active records
    order_summary_df = spark.table(f"{TARGET_CATALOG}.{TARGET_SCHEMA}.{ORDER_SUMMARY_TABLE}")
    .filter("IsActive = true")

```

```

# Aggregate total amount by customer name and date
aggregated_df = (
    order_summary_df
    .groupBy("Name", "Date")
    .agg(F.sum("TotalAmount").alias("TotalAmount"))
)

# Write to Delta table
aggregated_df.write.format("delta")
    .mode("overwrite")
    .saveAsTable(f"{TARGET_CATALOG}.{TARGET_SCHEMA}.{CUSTOMER_AGGREGATE_SPEND_TABLE}")

return aggregated_df

```

Step 6: Run the Pipeline

This step runs the entire batch processing pipeline. It creates a Spark session, processes the customer and order data, creates or updates the SCD Type 2 order summary table, and creates the customer aggregate spend table.

```

def run_pipeline():
    """Run the entire batch processing pipeline."""
    spark = create_spark_session()

    # Process customer and order data
    customer_df = process_customer_data(spark)
    order_df = process_order_data(spark)

    # Create or update SCD Type 2 order summary
    order_summary_df = create_order_summary_scd2(spark, customer_df,
    order_df)

    # Create customer aggregate spend
    customer_aggregate_df = create_customer_aggregate_spend(spark)

    return {
        "customer": customer_df,
        "order": order_df,
        "order_summary": order_summary_df,
        "customer_aggregate": customer_aggregate_df
    }

if __name__ == "__main__":
    run_pipeline()

```